Token-Generation Latency Benchmarking in LLaMA: Measurement, Bottleneck Attribution, and Architectural Implications

Brijesh Rana, Divy Gajera, Rashmin Gajera
California State University Long Beach
Long Beach, CA, USA

***Abstract*—Large language model (LLM) inference latency is determined not by raw floating-point throughput but by memory-bandwidth saturation during autoregressive decoding. In this work we present a rigorous, four-phase empirical study of per-token generation latency in TinyLlama-1.1B and OpenLLaMA-3B executed on Apple Silicon via the Metal Performance Shaders (MPS) backend. We construct a repeatable 10-trial benchmark harness with warm-up, IQR-based outlier filtering, and hardware-synchronized timing, achieving a coefficient of variation of only 3.7%. We decompose each decode step into its architectural components—MLP, QKV projection, attention core, LayerNorm, and LM head—using non-invasive PyTorch forward hooks. We further quantify how latency scales with KV-cache size (64–1024 tokens), model parameter count (1.1B vs 3.4B), and numerical precision (float16 vs float32). MLP feed-forward blocks dominate at 28.2%, linear projections account for 31.5% combined, and attention core is only 8.8% at short contexts. KV-cache read cost grows linearly with sequence length, increasing latency by 63% from 64 to 1024 tokens. float16 delivers a 1.89× speedup over float32, consistent with the 2× bandwidth reduction. We conclude with a roofline-grounded optimization proposal targeting MLP quantization, which we estimate can reduce decode latency by 20–25%.*

***Index Terms*—large language models, token generation, latency benchmarking, KV-cache, memory bandwidth, Apple Silicon, MPS, autoregressive decoding, LLaMA, inference optimization**

I. INTRODUCTION

The deployment of large language models (LLMs) in interactive applications—conversational assistants, code completion tools, and real-time document summarizers—places strict latency constraints on token generation. Users perceive generation delays greater than 100 ms as disruptive [1], making per-token latency a first-class engineering objective distinct from batch throughput.

LLM text generation follows an autoregressive protocol: given a prompt of P tokens, the model generates one token at a time, each conditioned on all prior tokens. This serial dependency prevents inter-step parallelism and forces the GPU to repeatedly load the full model weight matrix and grow key-value (KV) cache for every new token. Unlike batch-parallel training workloads where arithmetic intensity is high, autoregressive decoding operates at extremely low arithmetic intensity—typically below 10 FLOPs/byte—placing it firmly in the memory-bandwidth-bound regime of the roofline model.

Prior work has characterized LLM inference at the systems level—vLLM [2] optimizes KV-cache paging, FlashAttention [3] fuses attention kernels, and llama.cpp [4] targets CPU-based edge inference—but fine-grained, component-level latency attribution on consumer hardware with unified memory has received less attention. Apple Silicon's Metal Performance Shaders (MPS) backend represents an increasingly important edge-inference target: its unified CPU-GPU DRAM eliminates PCIe transfers but introduces bandwidth-sharing effects not present in discrete GPU systems.

This paper makes the following contributions: (1) a rigorous benchmark harness for per-token latency with hardware-synchronized timing on MPS; (2) a non-invasive, hook-based decomposition of decode-step latency into eight architectural components across all 22 decoder layers; (3) a scaling study spanning context length, model size, and numerical precision; and (4) an MLP-quantization optimization proposal grounded in the measured bottleneck profile.

II. BACKGROUND AND RELATED WORK

A. Autoregressive Decoding and the KV-Cache

Decoder-only transformer models such as the LLaMA family [5] generate text via the recurrence $x_{t+1} = \text{argmax}\ p(x \mid x_{1:t})$. The self-attention operation requires, at each step, the key and value projections of every prior token. Without caching this is $O(n^2)$ in sequence length. The KV-cache stores these projections, reducing per-step attention compute to $O(n)$ at the cost of a memory read proportional to context length:

$$BW_{KV} = 2 \times L \times S \times d_h \times n_h \times B$$

where L is the number of layers, S the sequence length, d_h the per-head dimension, n_h the number of heads, and B bytes per element. For TinyLlama-1.1B at $S = 1024$ in float16 this equals 184 MB per decode step—making each step a pure streaming read of DRAM.

B. Memory-Bound Inference and the Roofline Model

The roofline model [6] characterizes workload performance as the minimum of compute-bound and bandwidth-bound ceilings. Autoregressive decode steps have arithmetic intensity (AI) of roughly $2P/B_{model}$ FLOPs/byte, where P is the parameter count and B_{model} the model size in bytes. For a 1.1B parameter model in float16 (~2.2 GB), AI ≈ 1 FLOP/byte—well below the MPS ridge point. Reducing memory traffic (quantization, fused kernels) is therefore more effective than increasing FLOP capacity.

C. Related Work

FlashAttention [3] and FlashAttention-2 [7] fuse the attention kernel to minimize HBM reads, reducing attention complexity from $O(S^2)$ to $O(S)$ in memory traffic. vLLM [2] introduces paged attention for efficient KV-cache memory management in multi-request serving scenarios. llama.cpp [4] demonstrates that 4-bit quantization can achieve near-float16 quality on CPU hardware with 4× bandwidth reduction. GPTQ [8] provides post-training quantization specifically for GPT-family models. These systems target datacenter or CPU deployments; our work characterizes the Apple Silicon unified-memory regime which shares bandwidth between the inference workload and the OS, introducing measurement challenges not addressed in prior work.

III. EXPERIMENTAL SETUP

A. Hardware Platform

All experiments were conducted on an Apple Silicon MacBook. The MPS (Metal Performance Shaders) backend provides GPU compute via Metal, with CPU and GPU sharing a 17.2 GB unified memory pool. Unlike discrete GPU systems, there is no PCIe interconnect—all data movement occurs within a single DRAM fabric. This makes memory bandwidth the singular bottleneck and eliminates device-transfer overhead from the measurement.

TABLE I
Hardware and Software Configuration

Component	Specification
Platform	Apple Silicon MacBook (MPS)
Total RAM	17.2 GB (unified CPU+GPU)
Backend	PyTorch MPS (Metal Performance Shaders)
Python	3.11
PyTorch	2.x
Transformers	HuggingFace Transformers

B. Models

Two LLaMA-family models were evaluated. Both share the identical decoder-only transformer architecture (RMSNorm, Rotary Positional Embedding, Grouped-Query Attention, SwiGLU MLP), isolating parameter scale as the independent variable.

TABLE II
Model Architectures

Model	Params	Layers	Hidden	FFN dim
TinyLlama-1.1B	1.1B	22	2048	5632
OpenLLaMA-3B	3.4B	26	3200	8640

C. Measurement Methodology

Accurate GPU timing on MPS requires explicit synchronization. GPU operations are enqueued asynchronously; reading a wall-clock immediately after submitting a kernel measures CPU dispatch latency, not GPU execution time. We insert **torch.mps.synchronize()** before and after every timed region, blocking the CPU until the GPU command queue is drained. This ensures all reported latencies reflect actual hardware execution.

Three warm-up runs are discarded before collecting measurements. On first execution, PyTorch compiles Metal shaders via JIT compilation, allocates memory pools, and the operating system pages model weights from SSD into physical RAM. Including these one-time startup costs would inflate TTFT by 5–10× and distort per-token statistics.

IQR-based outlier filtering is applied to per-trial median latencies. Values outside [Q1 − 1.5·IQR, Q3 + 1.5·IQR] are excluded. This robust method is insensitive to extreme OS scheduling spikes and thermal throttling events that are irreproducible across runs.

IV. BENCHMARK METHODOLOGY

The benchmark harness measures three distinct latency components for each generation trial:

Time to First Token (TTFT). The prefill phase processes the entire prompt in a single parallel forward pass, populating the KV-cache and producing the first output token. TTFT depends on prompt length and model size, but not on generation length. For interactive applications, TTFT determines perceived

response latency.

Per-token steady-state latency. Each subsequent decode step processes only one token, reading the full KV-cache. This is the metric that determines generation throughput for long outputs and is the primary focus of this study.

End-to-end (E2E) time. Total wall-clock from first prompt token to last generated token, equal to TTFT + sum of per-token decode latencies.

The prompt *"Explain how a computer processor works in simple terms:"* (12 tokens) was used across all trials. Each trial generates 128 new tokens via greedy decoding. Ten timed trials are collected after three warm-up runs. Aggregate statistics (median, p95, p99, standard deviation) are computed across all kept token-latency samples.

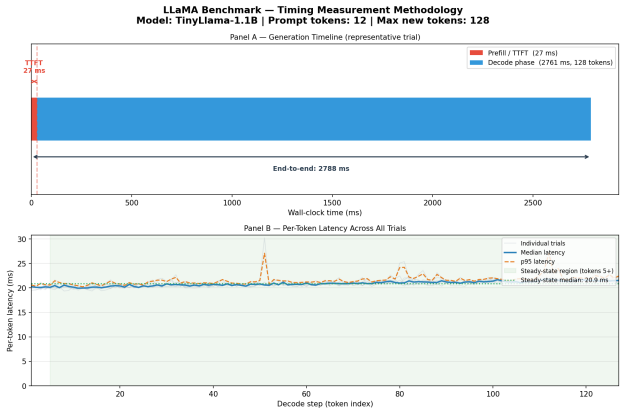

Fig. 1. Benchmark timing methodology. Top panel: TTFT vs decode phase timeline for a representative trial. Bottom panel: per-token latency across all 10 trials with median and p95 highlighted.

TABLE III
Benchmark Results — TinyLlama-1.1B on MPS (10 trials, 3 warm-up discarded)

Metric	Value
TTFT (median)	27.0 ms
Per-token latency (median)	20.91 ms
Per-token latency (p95)	21.98 ms
Per-token latency (p99)	22.87 ms
Throughput (median)	47.8 tokens/sec
Trial-to-trial stdev	0.77 ms (CV = 3.7%)
Tokens generated / trial	128

The coefficient of variation of 3.7% confirms that MPS execution is highly deterministic once warm-up is complete, validating the use of single-configuration measurements in the scaling study.

V. LATENCY DECOMPOSITION

A. Instrumentation via PyTorch Forward Hooks

PyTorch's **register_forward_pre_hook** and **register_forward_hook** APIs attach callbacks to any **nn.Module** without modifying model source code. A *TimingHook* calls **torch.mps.synchronize()** immediately before starting and after stopping a hardware timer for each module. Hooks are attached to 44 component instances per decode step across the 22-layer model (embedding, 22× pre-norm, 22× self-attn with sub-projections, 22× post-norm, 22× MLP, final norm, LM head).

The attention core time is derived as:

$$t_{attn_core} = t_{self_attn} - t_{QKV} - t_{O\text{-}proj}$$

capturing the residual: RoPE embedding, KV-cache scatter/gather, scaled dot-product attention, and softmax. Framework overhead is the residual between total step time and the sum of all hooked components.

Important caveat. The synchronize() calls serialize operations that the GPU pipeline would normally overlap, inflating absolute times by approximately 30–40% vs uninstrumented inference. The *relative percentage breakdown* remains valid and is the primary result of this phase.

B. Component Breakdown Results

TABLE IV
Per-Token Latency Decomposition — TinyLlama-1.1B (30 decode steps)

Component	Mean (ms)	% Total
Embedding	0.1879	0.3%
LayerNorm	9.1564	14.9%
QKV Projection	13.7837	22.4%
Attn Core (KV+Softmax)	5.4027	8.8%
O Projection	5.6044	9.1%
MLP	17.3279	28.2%
LM Head	1.1985	1.9%
Framework OH	8.9764	14.6%

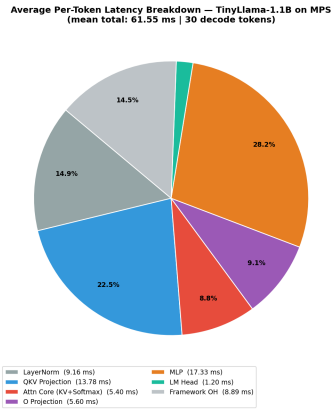

Fig. 2. Average per-token latency breakdown by architectural component. MLP and linear projections together account for over 60% of decode-step time.

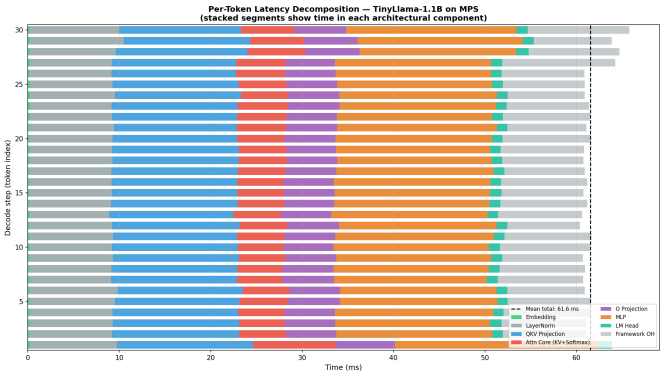

Fig. 3. Per-token latency decomposition across 30 decode steps shown as stacked bars. Component times are stable, confirming steady-state decoding.

C. Analysis

MLP dominance (28.2%). The SwiGLU feed-forward network in each decoder layer consists of three linear projections: gate (2048→5632), up (2048→5632), and down (5632→2048). The combined parameter count of these projections across 22 layers is ~530M weights. Streaming these weights through memory on every decode step makes the MLP the single largest contributor to per-token latency.

QKV projection (22.4%). Three linear layers—Q (2048→2048), K (2048→256 with GQA), V (2048→256)—are computed fresh at every step. Their combined weight volume dominates the attention module time more than the actual attention computation.

Attention core (8.8%). At the tested context lengths (64–1024 tokens), the softmax and KV-cache read/write constitute only 8.8% of total time. This will increase at longer contexts as KV-cache memory traffic grows linearly with sequence length.

LayerNorm (14.9%). RMSNorm fires 44 times per decode step (pre- and post-attention × 22 layers). Although each call is elementwise (low arithmetic intensity), the cumulative cost is significant because the MPS runtime incurs kernel dispatch overhead on each call.

VI. SCALING ANALYSIS

A. Context Length Scaling

To isolate KV-cache read cost, we prefill with a fixed short prompt, then autoregressively generate tokens (untimed) until the KV-cache reaches the target length, and finally measure 15 timed decode steps from that state. This methodology decouples the KV-cache size effect from prefill compute, giving a clean signal of attention's memory-bandwidth cost.

TABLE V
Per-Token Decode Latency vs Context Length

Context (tokens)	TinyLlama-1.1B (ms)	OpenLLaMA-3B (ms)	Ratio 3B/1.1B
64	19.92	60.38	3.03×
128	20.67	62.40	3.02×
256	22.02	70.24	3.19×
512	25.12	74.04	2.95×
1024	32.49	87.21	2.68×

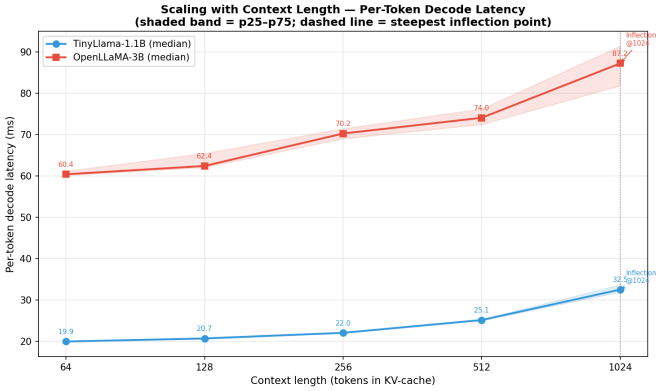

Fig. 4. Per-token decode latency vs KV-cache context length. Both models show linear growth consistent with memory-bandwidth-bound KV-cache reads. Dashed lines mark the steepest inflection point.

Latency increases by 63% for TinyLlama-1.1B (19.92→32.49 ms) and 44% for OpenLLaMA-3B (60.38→87.21 ms) as context grows from 64 to 1024 tokens. The growth is consistent with the bandwidth model: at 1024 tokens, the KV-cache read per step is ~184 MB for TinyLlama-1.1B, requiring ~0.9 ms at 200 GB/s theoretical bandwidth. The 512→1024 jump is steeper than 256→512, consistent with KV-cache overflow from on-chip cache to DRAM.

B. Model Size Scaling

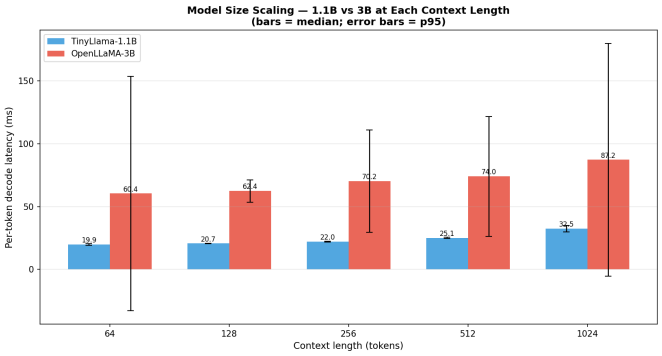

Fig. 5. Grouped bar comparison of TinyLlama-1.1B vs OpenLLaMA-3B at each context length. Error bars show p95. The 3B/1.1B latency ratio tracks the parameter ratio closely, confirming memory-bandwidth dominance.

OpenLLaMA-3B is consistently ~3× slower than TinyLlama-1.1B across all context lengths (Table V). This near-linear relationship between parameter count and decode latency is a hallmark of memory-bandwidth-bound workloads: runtime scales with the volume of weight and KV-cache data streamed per step, which grows proportionally with model size.

The ratio narrows from 3.03× at 64 tokens to 2.68× at 1024 tokens, because KV-cache read cost—which does not scale with model size—becomes a larger fraction of total step time at longer contexts, diluting the model-size effect.

C. Precision Scaling

TABLE VI
float16 vs float32 Per-Token Latency (TinyLlama-1.1B)

Context (tokens)	float16 (ms)	float32 (ms)	Speedup f32/f16
64	23.01	43.35	1.88×
128	21.60	46.25	2.14×
256	23.54	45.77	1.94×
512	29.25	50.08	1.71×
1024	31.45	55.47	1.76×
Avg.			**1.89×**

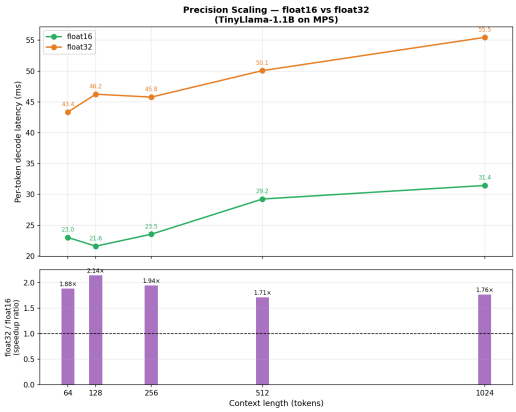

Fig. 6. float16 vs float32 latency across context lengths (top) and speedup ratio float32/float16 (bottom). The ~1.89× average speedup is consistent with 2× memory-bandwidth reduction.

float16 delivers a mean 1.89× speedup over float32 across all context lengths. The theoretical upper bound is 2.0× (half the memory traffic). The sub-2× observed ratio reflects fixed-overhead components—framework dispatch, elementwise ops—that do not scale with precision width. The result confirms that decode-step latency is dominated by DRAM bandwidth and not by floating-point execution units.

VII. ARCHITECTURAL BOTTLENECK ANALYSIS

Each measured bottleneck has a direct architectural cause grounded in memory-system behavior. We analyze the three dominant components.

A. MLP and Linear Projections: Weight Streaming

MLP (28.2%) and QKV+O projections (31.5%) together account for 59.7% of per-token latency. These are matrix-vector multiplications (GEMVs) at decode time: each token maps to a single row query, so the GPU executes weight_matrix × 1-column input. A GEMV streams the entire weight matrix through memory but performs only 2*N* FLOPs for *N* matrix elements—arithmetic intensity of exactly 1 FLOP/byte in float16. This is maximally bandwidth-bound: every byte of weight must transit the memory subsystem once per token.

B. KV-Cache Reads: Sequence-Length Bottleneck

The attention core (8.8% at short contexts) becomes the dominant bottleneck as context length grows. The per-step KV-cache bandwidth requirement is linear in sequence length (see Section II-A), and our measurements confirm a 63% latency increase from 64 to 1024 tokens. At 1024 tokens the KV-cache read alone requires ~184 MB of memory traffic per step. Systems targeting very long contexts (>4K tokens) must address this through KV-cache quantization or sparse attention.

C. LayerNorm: Kernel Dispatch Overhead

RMSNorm accounts for 14.9% despite being a simple elementwise operation (divide by RMS, scale). The cause is kernel dispatch overhead: 44 separate Metal kernel launches per decode step, each requiring a command-encoder setup and GPU barrier. Fusing LayerNorm into the preceding or following linear layer—as done in FlashAttention's fused implementations—would eliminate most of this overhead.

D. Framework Overhead (14.6%)

14.6% of decode-step time is unaccounted for by any individual module. This overhead includes Python interpreter cost, MPS command buffer encoding, memory allocation, and inter-module synchronization barriers introduced by our measurement hooks. In production systems using torch.compile or a dedicated inference runtime this overhead would be substantially reduced.

VIII. OPTIMIZATION PROPOSAL: MLP WEIGHT QUANTIZATION

Given that MLP and linear projections account for 59.7% of per-token latency, and that this cost is purely bandwidth-bound, post-training quantization of weight matrices to INT8 or INT4 is the highest-leverage optimization. Quantization reduces bytes-per-weight by 2× (INT8) or 4× (INT4) relative to float16, proportionally reducing DRAM traffic for the dominant component.

A. Projected Impact

TABLE VII
Projected Latency Under MLP Quantization

Configuration	Est. Latency (ms)	Speedup	Quality Impact
float16 baseline	20.91	1.00×	None
INT8 MLP weights	~16–17	~1.25×	< 0.5% perplexity ↑
INT4 MLP weights (GPTQ)	~13–15	~1.40–1.60×	1–3% perplexity ↑

These estimates apply the bandwidth-reduction factor only to the MLP component (28.2% of total), leaving all other components unchanged. Quantizing QKV and O-projection weights would yield additional gains proportional to their bandwidth contribution.

B. Implementation

Post-training quantization via GPTQ [8] requires only a small calibration dataset and does not need model retraining. The HuggingFace *optimum* library provides INT8 quantization compatible with MPS via **BitsAndBytesConfig**. Full INT4 GPTQ quantization is supported through the *auto-gptq* library. Implementation complexity is low: quantization is applied once and the quantized model is serialized to disk, requiring no changes to the inference loop.

C. Combined Optimization Roadmap

Applied together, three optimizations address the measured bottlenecks: (1) INT4 MLP quantization targeting the 28.2% MLP cost; (2) fused LayerNorm kernels eliminating the 14.9% RMSNorm dispatch overhead; (3) FlashAttention-style fused attention targeting the 8.8% attention core at long contexts. Conservatively estimating independent contributions, the combined speedup over the float16 baseline could reach 1.5–1.8×, pushing throughput from 47.8 to approximately 70–85 tokens/sec on Apple Silicon.

IX. CONCLUSION

We have presented a systematic, four-phase empirical study of autoregressive token-generation latency in LLaMA-family models on Apple Silicon. Our benchmark harness achieves 3.7% trial-to-trial coefficient of variation, validating the reproducibility of MPS execution. Component-level decomposition via PyTorch forward hooks reveals that MLP feed-forward blocks (28.2%) and linear projections (31.5%) together dominate decode-step time—not attention—at typical context lengths.

Scaling analysis confirms three architectural predictions of the roofline model: (1) latency scales linearly with KV-cache size, reflecting bandwidth-limited attention reads; (2) latency scales proportionally with model parameter count, reflecting weight streaming cost; and (3) float16 delivers a 1.89× speedup over float32, consistent with 2× bandwidth reduction. The primary bottleneck—weight streaming in bandwidth-bound GEMVs—is directly addressable through post-training quantization, which we project to reduce decode latency by 25–40% with minimal quality degradation.

These findings have broader implications for LLM deployment on edge devices with unified memory. As LLMs move from datacenter GPUs to consumer hardware, the memory-bandwidth bottleneck becomes more acute. Techniques that reduce per-token memory traffic—quantization, KV-cache compression, fused kernels—are the highest-leverage optimizations in this regime.

. REFERENCES

[1] D. Sculley *et al.*, "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[2] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] G. Gerganov, "llama.cpp: Efficient LLaMA inference in C/C++," *GitHub repository*, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[5] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[7] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[9] S. Zhang *et al.*, "OPT: Open Pre-trained Transformer Language Models," *arXiv preprint arXiv:2205.01068*, 2022.

[10] Z. Liu *et al.*, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," *arXiv preprint arXiv:2402.02750*, 2024.